

FIM1START et le vecteur IM1_CALLBACK

FIM1START — installation du vecteur

```
; -----  
; ROUTINE : F_IM1_START  
; Rôle    : Installe F_IM1_CALLBACK comme cible du vecteur  
;          d'interruption IM1 (adresse #0038, où le firmware  
;          CPC place un JP nnnn au démarrage). N'écrit que  
;          l'opérande de ce JP (#0039/#003A), pas l'octet JP  
;          lui-même déjà en place.  
; Entrée  : aucune  
; Sortie  : (#0039) = adresse de F_IM1_CALLBACK  
; Détruit : AF, HL  
; -----  
F_IM1_START  
    DI                                ; Désactive le temps de la modif (non-atomique)  
    LD    HL, F_IM1_CALLBACK  
    LD    (#0039), HL  
    EI                                ; Réactive globalement les interruptions  
    RET
```

Sur CPC, l'adresse `#0038` contient toujours `JP nnnn` (placé là par le firmware au démarrage) — le CPU y saute automatiquement à chaque interruption matérielle en mode IM1. Cette routine n'écrit **que l'opérande** de ce `JP` (aux adresses `#0039` / `#003A`), redirigeant le saut vers `F_IM1_CALLBACK` sans toucher à l'octet `JP` (`#C3`) lui-même déjà en place.

Le DI — pourquoi il est indispensable

`LD (#0039), HL` écrit deux octets en mémoire, donc n'est pas atomique. Si une interruption survient pile entre l'écriture de l'octet bas et celle de l'octet haut, le CPU sauterait sur une adresse hybride (moitié ancienne, moitié nouvelle) — un vrai risque de plantage. Le `DI` élimine ce risque en désactivant les interruptions le temps de la modification, ce qui est important puisque `F_IM1_START` peut être appelée en cours de jeu (via `F_SCENE_CHARGER`), pas seulement une fois au démarrage.

FIM1CALLBACK — le cœur de la synchronisation

```
; -----  
; ROUTINE : F_IM1_CALLBACK  
; Rôle    : Point d'entrée de l'interruption IM1 (~300 Hz,  
;          cadencée par le CRTC). Détecte le vrai retour  
;          vertical (VBL) via le port PPI plutôt que de  
;          compter aveuglément les interruptions, ce qui  
;          corrige toute dérive du timing CRTC et cale le  
;          jeu sur exactement 50 Hz. Si un VBL est détecté,  
;          libère FRAME_READY (débloque BOUCLE_PRINCIPALE).  
;          Sinon, avance IM1_INDEX et dispatche vers la  
;          sous-tâche IM1 courante via la table IM1_CURRENT  
;          (mise à jour par F_SCENE_CHARGER selon la scène  
;          active).  
; Entrée  : aucune (déclenchée par le matériel)  
; Sortie  : FRAME_READY = 0 si VBL détecté  
;          IM1_INDEX mis à jour (avancé ou réinitialisé)  
;          saut final vers la sous-routine IM1 correspondante  
;          (JP (HL)), ou vers F_IM1_SKIP si IM1_INDEX n'est  
;          pas encore prêt (IM1_NOT_READY)  
; Détruit : rien de visible pour l'appelant – sauvegarde et  
;          restaure intégralement AF, BC, DE, HL, IX, IY  
;          (registres principaux et alternatifs) avant RETI  
; Dépend  : F_IM1_SKIP, IM1_CURRENT, IM1_INDEX, IM1_IT_MAX,  
;          IM1_NOT_READY, FRAME_READY  
; -----  
F_IM1_CALLBACK  
    ; SAUVEGARDE DU CONTEXTE  
    EXX  
    EX    AF, AF'  
    PUSH  IX  
    PUSH  IY  
  
    ; DETECTER SYNCHRO VBL  
    LD    A, HI(PPI_PORT_B)  
    IN    A, (0)  
    RRCA  
    JR    NC, .NO_VBL  
  
    ; DEBUT DE FRAME (Fréquence exacte de 50 Hz)  
    LD    HL, FRAME_READY
```

```

XOR    A
LD      (HL), A
JR      .DISPATCH

.NO_VBL
LD      A, (IM1_INDEX)
CP      IM1_NOT_READY
JR      Z, F_IM1_SKIP      ; pas de VBL et pas encore prêt -> sortie

INC     A
CP      IM1_IT_MAX
JR      C, .DISPATCH
XOR     A

.DISPATCH
LD      (IM1_INDEX), A
ADD     A, A
LD      C, A
LD      B, 0
LD      HL, (IM1_CURRENT)
ADD     HL, BC
LD      A, (HL)
INC     HL
LD      H, (HL)
LD      L, A
JP      (HL)

```

Trois responsabilités en une seule routine

1. **Sauvegarde de contexte complète** (`EXX`, `EX AF,AF'`, `PUSH IX/IY`) — indispensable puisque cette routine interrompt du code arbitraire, y compris du code qui pourrait déjà utiliser les registres alternatifs.
2. **Détection du vrai VBL** — relit le port PPI à chaque appel (~300 Hz), corrigeant toute dérive du timing CRTIC plutôt que de compter aveuglément les interruptions.
3. **Dispatch vers la sous-tâche IM1 courante** — via `IM1_CURRENT`, une table de vecteurs mise à jour par `F_SCENE_CHARGER` à chaque changement de scène, permettant à chaque scène (menu, jeu, game over) d'avoir son propre découpage des 6 sous-tâches par frame.

La factorisation avec `FIM1SKIP`

Le `JR Z, F_IM1_SKIP` saute directement vers une routine externe qui restaure le contexte et sort (`POP IY/IX`, `EX AF,AF'`, `EXX`, `EI`, `RETI`) — évite de dupliquer ces 9 octets à deux endroits (l'ancien bloc `.EXIT` en fin de routine a été supprimé au profit de ce saut).

Comment les deux routines s'articulent

```
F_IM1_START (une fois au démarrage, ou à chaque F_SCENE_CHARGER)
    -> écrit #0039 pour rediriger #0038 -> F_IM1_CALLBACK

Ensuite, ~300 fois par seconde :
    Interruption matérielle -> JP #0038 (firmware) -> JP F_IM1_CALLBACK
        -> sauvegarde contexte
        -> teste le vrai VBL
            si VBL : FRAME_READY = 0 (débloque BOUCLE_PRINCIPALE)
            sinon  : avance IM1_INDEX, dispatch vers la sous-tâche courante
        -> restaure contexte, RETI
```

C'est cette chaîne complète qui permet à `BOUCLE_PRINCIPALE` (dans `F_Z80LIB_LANCER`) de rester bloquée en attente jusqu'au bon moment exact, 50 fois par seconde, tout en laissant de la place pour des sous-tâches IM1 intermédiaires (son, animations secondaires, etc.) réparties sur les ~300 Hz disponibles entre deux VBL.

Principe important : garder l'ISR minimale

Cette routine ne doit contenir que ce qui est strictement nécessaire au dispatch et à la synchro. Tout traitement supplémentaire (compteur de frames, timing de jeu, animations) doit être déplacé vers un niveau plus approprié — idéalement dans la routine de scène elle-même (appelée depuis `BOUCLE_PRINCIPALE` à 50 Hz), pas dans l'ISR qui tourne, elle, à ~300 Hz et doit rester la plus légère possible pour ne pas retarder le dispatch des autres sous-tâches IM1.